

claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\agent-ad8c274c7cc27ec66.jsonl`  
- SHA-256: `782d6f7487ee7a07ac0daf462b138ca0c407c72c22d7cafc4ff7f11485a0142b`  
- Source modified: `2026-07-03T17:37:30+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `subagents`  
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`
```

Transcript

1. user (2026-07-03T17:33:21.156Z)

You are a code-review finder agent. Review target: PR #452 of badminton-highlight-indexer at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, branch codex/a2-golden-manifest-portability (HEAD 5a91701). Diff: git diff origin/master..HEAD. The PR replaces raw `json.load(manifest)` with a STRICT `load\_resolved\_manifest` (raises ManifestPathError when any required trajectory/labels file is missing) across 8 consumers, and rewrites tests/test_served_gate_a_regression._golden_specs.

ANGLE B ? removed-behavior audit. For every deleted/replaced line, name the invariant or tolerance it provided and check where the new code re-establishes it. Focus on:

1. tests/test_served_gate_a_regression.py _golden_specs: OLD code filtered per-video (kept the subset of specs whose files exist, ran if >=6 survived). NEW code strict-loads ALL: one missing file anywhere -> ManifestPathError -> [] -> every test skips. Is that a coverage regression (a box/CI with a partial corpus silently loses the whole regression suite instead of running on the >=6 subset)? Read the module docstring/comments for the intended gating semantics; check what MANIFEST points at and when this runs (owner box only?).
2. Each consumer (calibrate_local, distill_local, fusion_golden, rally_seq_proto, serve_contrast, served_gate_a, gen0/harness, rally_seq_eval): did any previously TOLERATE missing files per-video (skip/shrink/lazy-fail later) in a way that was load-bearing for some caller (e.g., nightly_regression calling rally_seq_eval.load_golden with features present but trajectories absent; fusion_golden --fresh resume flows)? Grep callers of each changed function (scripts/nightly_regression.py especially) and decide whether strict-by-default breaks a real usage.
3. Removed `import json` lines: confirm via grep that each file has no remaining json usage (served_gate_a.py especially ? check whether its main()/CLI prints json).
4. tests/test_fusion_compare.py and test_fusion_golden_telemetry.py: the PR adds file-creation lines to make old tests pass under the strict loader ? did any test LOSE what it was originally asserting (e.g., a test that previously exercised the missing-file tolerance)?

Return up to 6 candidates as JSON [{file, line, summary, failure_scenario}]. [] if none. Final message = only the JSON array.

2. assistant (2026-07-03T17:33:27.084Z)

I'll analyze this PR for removed-behavior regressions. Let me start by understanding the diff and the key changes.

3. user (2026-07-03T17:33:28.036Z)

5a91701 eval: add portable golden manifest check

4. assistant (2026-07-03T17:33:30.120Z)

Now let me check the git diff to understand what changed:

5. user (2026-07-03T17:33:31.015Z)

```
backend/eval/calibrate_local.py      | 5 +-
backend/eval/distill_local.py        | 5 +-
backend/eval/fusion_golden.py        | 4 +-
backend/eval/golden_manifest.py      | 378 +++++
backend/eval/rally_seg_eval.py       | 4 +-
backend/eval/rally_seg_proto.py      | 5 +-
backend/eval/serve_contrast.py       | 4 +-
backend/eval/served_gate_a.py        | 5 +-
docs/ALPHA_LAUNCH_READINESS.md      | 11 +-
docs/data_pipeline/GOLDEN_VIDEOS.md | 8 +-
tests/test_fusion_compare.py         | 4 +
tests/test_fusion_golden_telemetry.py | 3 +
tests/test_golden_manifest.py        | 158 +++++
tests/test_served_gate_a_regression.py | 21 +-
training/gen0/harness.py             | 4 +-
15 files changed, 586 insertions(+), 33 deletions(-)
```

6. assistant (2026-07-03T17:33:33.366Z)

Let me check the test_served_gate_a_regression.py file to understand the OLD behavior:

7. user (2026-07-03T17:33:34.187Z)

""SERVED Gate-A regression litmus ? the *production* FusionSegmenter path on real footage.

The flagged gap (#146): the served default flip (`default_segmenter=fusion_hybrid` +`min_crossings=2``) is **mock-tested only** on real video (`test_fusion_hybrid.py`` drives a MagicMock runner). The offline ablation litmus (`test_ablation.test_litmus_reproduces_gate_a``) validates the Gate-A **signal** ? but on the **permissive proposal windowing** (`distill_local.PROPOSAL``), NOT the production operating point. This module closes the gap: it runs the real `FusionSegmenter`` decision seams (`enrich_candidate_stats`` net-crossings + `select_for_handoff`` Gate A) over windows from the **production** `trajectory_to_action_windows`` config, on the cached golden WASB trajectories, and asserts the served path's measured behaviour.

GPU-free / Gemini-free (person cue OFF) ? it stops at the AI-handoff boundary. Skips cleanly in CI where the golden trajectories are absent; runs on the owner box where they are present, over **whatever golden clips are currently on disk** (`golden_specs`` requires ≥ 6).

CORPUS NOTE ? re-grounded 2026-06-22 (the golden set is mutable and grows over time; this is the SECOND re-grounding ? #236 pinned the 2026-06-18 9-clip snapshot, which the corpus has since outgrown). The original 6 **single-court** golden clips this litmus was first calibrated on (2026-06-14) have been retired from the box ? their labels are gone ? so the present golden set is

the **15 multicourt clips** now on disk (mahadevpura_1/2/singles, GX0101xx, GX0x0094, Badminton_BXH_2, Boxhill_Doubles, adarsh_avi_singles, kushagra_singles, largetest_doubles, gbaaddy, testlarge_short). On that corpus the served path is **recall-starved**: WASB tracks a single shuttle

while several courts rally at once, so `trajectory_to_action_windows`` emits FEWER windows than there are rallies (served over-seg ratio < 1 , i.e. under-segmentation). Because Gate A only ever DROPS windows, on this under-segmented corpus it is mildly **F1-negative** (mean served ?F1 ? -0.023), not the ~neutral -0.008 the original 6 showed. Under the permissive PROPOSAL windowing the same gate is ~F1-neutral (?F1 ? +0.03) while still cutting over-segmentation (1.36?0.79).

The canonical **+0.074 / 6-of-6 / p=0.031** Gate-A win was a property of the **original-6** corpus and is NOT reproduced here (those clips are no longer on the box); it is preserved in the archive

(`docs/archives/quality-iterations/2026-06-first-ab-experiment/``). These tests now pin the **present-corpus** behaviour so a future windowing/corpus change that silently shifts it trips the

litmus. The pinned magnitudes are a date-stamped snapshot of the present golden set and will

```

need
re-measurement as the corpus grows.
"""

```

```

from __future__ import annotations

```

```

import os

```

```

import pytest

```

cv2 is imported transitively by trajectory_hybrid (the served engine). On a box without (CI), skip the whole module rather than erroring at import.

```

pytest.importorskip("cv2")

```

```

from backend.eval import segmentation_metrics as _segm # noqa: E402
from backend.eval import served_gate_a as sga # noqa: E402
from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
from backend.eval.golden_manifest import ( # noqa: E402
    ManifestPathError,
    load_resolved_manifest,
)
from backend.eval.metrics import score # noqa: E402
from backend.pipeline.detectors import rally_gate # noqa: E402
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402

```

The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a fresh worktree / CI). Resolve it relative to this repo root first, with an env override for an out-of-tree checkout.

```

_REPO0_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
MANIFEST = os.environ.get(
    "SERVED_GATE_A_MANIFEST",
    os.path.join(_REPO0_ROOT, "output", "human_lovo_manifest.json"),
)

```

```

def _golden_specs():
    """Return rebased manifest specs when trajectory + label files are present, else []."""
    if not os.path.isfile(MANIFEST):
        return []
    try:
        specs = load_resolved_manifest(MANIFEST)
    except ManifestPathError:
        return []
    return specs if len(specs) >= 6 else []

```

```

_SPECS = _golden_specs()
_skip = pytest.mark.skipif(
    not _SPECS,
    reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
)

```

---- #398 regression: golden-free (runs in CI; only the golden DATA is absent there, no

```

def test_served_idx_cfg_is_typed_and_seams_accept_it(tmp_path):
    """The served litmus must pass a TYPED IndexingConfig (what production passes), not a dict ?
    `_enrich_candidate_stats` reads `idx_cfg.fusion`, so a dict raised AttributeError (#398).
    Build
    the config + drive the full served seam path on a synthetic trajectory (no golden data
    needed)."""
    from backend.config.models import IndexingConfig

```

```

cfg = sga._served_idx_cfg(2)
assert isinstance(cfg, IndexingConfig)
assert cfg.fusion.min_crossings == 2
traj = tmp_path / "synth.csv"
rows = ["Frame,Visibility,X,Y"]
for i in range(120): # oscillate across the net so windows + crossings form
    x = 200 + (i % 20) * 40

```

8. assistant (2026-07-03T17:33:36.494Z)

Now let me see the new version:

9. user (2026-07-03T17:33:37.199Z)

```

1      """SERVED Gate-A regression litmus ? the *production* FusionSegmenter path on real
footage.
2
3      The flagged gap (#146): the served default flip (`default_segmenter=fusion_hybrid` +
4      ``min_crossings=2``) is **mock-tested only** on real video (`test_fusion_hybrid.py`
drives a
5      MagicMock runner). The offline ablation litmus
6      (`test_ablation.test_litmus_reproduces_gate_a`)
7      validates the Gate-A *signal* ? but on the **permissive proposal windowing**
8      (`distill_local.PROPOSAL`), NOT the production operating point. This module closes the
gap: it
9      runs the real ``FusionSegmenter`` decision seams (`enrich_candidate_stats` net-
crossings +
10     ``select_for_handoff`` Gate A) over windows from the **production**
11     ``trajectory_to_action_windows`` config, on the cached golden WASB trajectories, and
asserts
12     the served path's measured behaviour.
13
14     GPU-free / Gemini-free (person cue OFF) ? it stops at the AI-handoff boundary. Skips
cleanly in
15     CI where the golden trajectories are absent; runs on the owner box where they are
present, over
16     **whatever golden clips are currently on disk** (`golden_specs` requires >= 6).
17
18     CORPUS NOTE ? re-grounded 2026-06-22 (the golden set is mutable and grows over time;
this is the
19     SECOND re-grounding ? #236 pinned the 2026-06-18 9-clip snapshot, which the corpus has
since
20     outgrown). The original 6 *single-court* golden clips this litmus was first calibrated
on
21     (2026-06-14) have been retired from the box ? their labels are gone ? so the present
golden set is
22     the **15 multicourt clips** now on disk (mahadevpura_1/2/singles, GX0101xx, GX0x0094,
23     Badminton_BXH_2, Boxhill_Doubles, adarsh_avi_singles, kushagra_singles,
largetest_doubles, gbaaddy,
24     testlarge_short). On that corpus the served path is **recall-starved**: WASB tracks a
single shuttle
25     while several courts rally at once, so ``trajectory_to_action_windows`` emits FEWER
windows than
26     there are rallies (served over-seg ratio < 1, i.e. under-segmentation). Because Gate A
only ever
27     DROPS windows, on this under-segmented corpus it is mildly **F1-negative** (mean served
?F1 ?
28     -0.023), not the ~neutral -0.008 the original 6 showed. Under the permissive PROPOSAL
windowing the
29     same gate is ~F1-neutral (?F1 ? +0.03) while still cutting over-segmentation
(1.36?0.79).
30
31     The canonical **+0.074 / 6-of-6 / p=0.031** Gate-A win was a property of the
*original-6* corpus
32     and is NOT reproduced here (those clips are no longer on the box); it is preserved in

```

```

the archive
32     (`docs/archives/quality-iterations/2026-06-first-ab-experiment/``). These tests now pin
the
33     **present-corpus** behaviour so a future windowing/corpus change that silently shifts it
trips the
34     litmus. The pinned magnitudes are a date-stamped snapshot of the present golden set and
will need
35     re-measurement as the corpus grows.
36     ""
37
38     from __future__ import annotations
39
40     import os
41
42     import pytest
43
44     # cv2 is imported transitively by trajectory_hybrid (the served engine). On a box
without it
45     # (CI), skip the whole module rather than erroring at import.
46     pytest.importorskip("cv2")
47
48     from backend.eval import segmentation_metrics as _segm # noqa: E402
49     from backend.eval import served_gate_a as sga # noqa: E402
50     from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
51     from backend.eval.golden_manifest import ( # noqa: E402
52         ManifestPathError,
53         load_resolved_manifest,
54     )
55     from backend.eval.metrics import score # noqa: E402
56     from backend.pipeline.detectors import rally_gate # noqa: E402
57     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402
58
59     # The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a
fresh
60     # worktree / CI). Resolve it relative to this repo root first, with an env override for
an
61     # out-of-tree checkout.
62     _REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
63     MANIFEST = os.environ.get(
64         "SERVED_GATE_A_MANIFEST",
65         os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
66     )
67
68
69     def _golden_specs():
70         """Return rebased manifest specs when trajectory + label files are present, else
[ ]."""
71         if not os.path.isfile(MANIFEST):
72             return []
73         try:
74             specs = load_resolved_manifest(MANIFEST)
75         except ManifestPathError:
76             return []
77         return specs if len(specs) >= 6 else []
78
79
80     _SPECS = _golden_specs()
81     _skip = pytest.mark.skipif(
82         not _SPECS,
83         reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
84     )
85
86
87     # ---- #398 regression: golden-free (runs in CI; only the golden DATA is absent there,
not cv2) ----

```

```

88     def test_served_idx_cfg_is_typed_and_seams_accept_it(tmp_path):
89         """The served litmus must pass a TYPED IndexingConfig (what production passes), not
a dict ?
90         `_enrich_candidate_stats` reads `idx_cfg.fusion`, so a dict raised AttributeError
(#398). Build
91         the config + drive the full served seam path on a synthetic trajectory (no golden
data needed)."""
92         from backend.config.models import IndexingConfig
93
94         cfg = sga._served_idx_cfg(2)
95         assert isinstance(cfg, IndexingConfig)
96         assert cfg.fusion.min_crossings == 2
97         traj = tmp_path / "synth.csv"
98         rows = ["Frame,Visibility,X,Y"]
99         for i in range(120): # oscillate across the net so windows + crossings form
100             x = 200 + (i % 20) * 40

```

10. assistant (2026-07-03T17:33:39.021Z)

The `_golden_specs()` looks the same. Let me check what the actual changes are to this file:

11. user (2026-07-03T17:33:39.999Z)

```

diff --git a/tests/test_served_gate_a_regression.py b/tests/test_served_gate_a_regression.py
index 7fd7630..56848df 100644
--- a/tests/test_served_gate_a_regression.py
+++ b/tests/test_served_gate_a_regression.py
@@ -37,7 +37,6 @@ re-measurement as the corpus grows.

```

```

from __future__ import annotations

import json
import os

import pytest
@@ -49,6 +48,10 @@ pytest.importorskip("cv2")
from backend.eval import segmentation_metrics as _segm # noqa: E402
from backend.eval import served_gate_a as sga # noqa: E402
from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
+from backend.eval.golden_manifest import ( # noqa: E402
+    ManifestPathError,
+    load_resolved_manifest,
+)
from backend.eval.metrics import score # noqa: E402
from backend.pipeline.detectors import rally_gate # noqa: E402
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402
@@ -64,18 +67,14 @@ MANIFEST = os.environ.get(

def _golden_specs():
-    """Return the manifest specs whose trajectory + label files are all present, else []."""
+    """Return rebased manifest specs when trajectory + label files are present, else []."""
    if not os.path.isfile(MANIFEST):
        return []
-    with open(MANIFEST, encoding="utf-8") as f:
-        specs = json.load(f)
-    ok = [
-        s
-        for s in specs
-        if os.path.isfile(str(s.get("trajectory", "")))
-        and os.path.isfile(str(s.get("labels", "")))
-    ]
-    return ok if len(ok) >= 6 else []
+    try:
+        specs = load_resolved_manifest(MANIFEST)

```

```

+     except ManifestPathError:
+         return []
+     return specs if len(specs) >= 6 else []

```

```
_SPECS = _golden_specs()
```

12. assistant (2026-07-03T17:33:41.642Z)

Good! This is the critical change. Let me check what `load_resolved_manifest` does:

13. user (2026-07-03T17:33:42.424Z)

```

1     """Portable golden-corpus manifest loader and smoke checker.
2
3     The owner-side golden manifest is intentionally gitignored and has historically
4     carried absolute paths from whichever box ingested a clip. This module keeps that
5     artifact usable after the workspace moves by resolving stale paths against the
6     current repo layout and failing loudly when required corpus files are absent.
7
8     Default local layout supported:
9     - ../Annotation Setup/Trajectories/<trajectory>.csv
10    - ../Annotation Setup/Golden Labelled/<labels>.csv
11    - output/<labels>.csv
12
13    Extra roots can be supplied with ``--root`` or ``RALLY_GOLDEN_SEARCH_ROOTS``.
14    """
15
16    from __future__ import annotations
17
18    import argparse
19    import json
20    import os
21    import sys
22    from dataclasses import dataclass
23    from pathlib import Path, PureWindowsPath
24    from typing import Any, Iterable, Sequence
25
26    DEFAULT_MANIFEST = "output/human_lovo_manifest.json"
27    SEARCH_ROOTS_ENV = "RALLY_GOLDEN_SEARCH_ROOTS"
28    DEFAULT_REQUIRED_FIELDS = ("trajectory", "labels")
29
30    _FIELD_DIR = {
31        "trajectory": "Trajectories",
32        "labels": "Golden Labelled",
33    }
34
35
36    @dataclass(frozen=True)
37    class ManifestIssue:
38        """One manifest path problem found by the resolver."""
39
40        name: str
41        field: str
42        path: str
43        reason: str
44
45        def message(self) -> str:
46            shown = f" {self.path!r}" if self.path else ""
47            return f"{self.name}: {self.field}{shown} - {self.reason}"
48
49
50    class ManifestPathError(RuntimeError):
51        """Raised when a strict manifest load finds missing required files."""
52

```

```

53     def __init__(self, manifest_path: str | os.PathLike[str], issues:
Sequence[ManifestIssue]):
54         self.manifest_path = os.fspath(manifest_path)
55         self.issues = list(issues)
56         detail = "\n".join(f" - {i.message()}" for i in self.issues[:12])
57         more = "" if len(self.issues) <= 12 else f"\n ... {len(self.issues) - 12} more"
58         super().__init__(
59             f"golden manifest path check failed for {self.manifest_path}: "
60             f"{len(self.issues)} issue(s)\n{detail}{more}"
61         )
62
63
64     def repo_root() -> Path:
65         """Return the repository root for this module."""
66
67         return Path(__file__).resolve().parents[2]
68
69
70     def load_manifest(manifest_path: str | os.PathLike[str]) -> list[dict[str, Any]]:
71         """Load the flat golden manifest list and validate its outer shape."""
72
73         path = Path(manifest_path)
74         with path.open(encoding="utf-8") as f:
75             data = json.load(f)
76         if not isinstance(data, list):
77             raise ValueError(f"golden manifest must be a JSON list: {path}")
78         out: list[dict[str, Any]] = []
79         for i, row in enumerate(data):
80             if not isinstance(row, dict):
81                 raise ValueError(f"golden manifest row {i} is not an object: {path}")
82             out.append(dict(row))
83         return out
84
85
86     def resolve_manifest_paths(
87         manifest_path: str | os.PathLike[str],
88         *,
89         repo_root_path: str | os.PathLike[str] | None = None,
90         extra_roots: Iterable[str | os.PathLike[str]] = (),
91         required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
92     ) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
93         """Load and rebase required path fields in a golden manifest.
94
95         Returns ``(resolved_specs, issues)``. The returned specs preserve all manifest
96         metadata but replace any resolved ``trajectory`` / ``labels`` path with the
97         existing local path. Missing required fields are reported in ``issues``.
98         """
99
100         manifest = load_manifest(manifest_path)
101         return resolve_manifest_specs(
102             manifest,
103             manifest_path,
104             repo_root_path=repo_root_path,
105             extra_roots=extra_roots,
106             required_fields=required_fields,
107         )
108
109
110     def load_resolved_manifest(
111         manifest_path: str | os.PathLike[str],
112         *,
113         repo_root_path: str | os.PathLike[str] | None = None,
114         extra_roots: Iterable[str | os.PathLike[str]] = (),
115         required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
116         strict: bool = True,

```

```

117 ) -> list[dict[str, Any]]:
118     """Load a manifest with stale paths rebased; optionally raise on missing files."""
119
120     specs, issues = resolve_manifest_paths(
121         manifest_path,
122         repo_root_path=repo_root_path,
123         extra_roots=extra_roots,
124         required_fields=required_fields,
125     )
126     if strict and issues:
127         raise ManifestPathError(manifest_path, issues)
128     return specs
129
130
131 def resolve_manifest_specs(
132     specs: Sequence[dict[str, Any]],
133     manifest_path: str | os.PathLike[str],
134     *,
135     repo_root_path: str | os.PathLike[str] | None = None,
136     extra_roots: Iterable[str | os.PathLike[str]] = (),
137     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
138 ) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
139     """Resolve an already-loaded manifest list."""
140
141     manifest = Path(manifest_path)
142     root = Path(repo_root_path).resolve() if repo_root_path else repo_root()
143     roots = [Path(p) for p in _env_roots()]
144     roots.extend(Path(p) for p in extra_roots)
145     required = tuple(required_fields)
146
147     resolved: list[dict[str, Any]] = []
148     issues: list[ManifestIssue] = []
149     for idx, spec in enumerate(specs):
150         row = dict(spec)

```

14. assistant (2026-07-03T17:33:44.488Z)

Let me see more of the resolution logic:

15. user (2026-07-03T17:33:45.367Z)

```

150         row = dict(spec)
151         name = str(row.get("name") or f"row[{idx}]")
152         for field in required:
153             raw = row.get(field)
154             if not raw:
155                 issues.append(ManifestIssue(name, field, "", "required field missing"))
156                 continue
157             hit = resolve_existing_path(
158                 str(raw),
159                 field,
160                 manifest_path=manifest,
161                 repo_root_path=root,
162                 extra_roots=roots,
163             )
164             if hit is None:
165                 issues.append(ManifestIssue(name, field, str(raw), "file not found"))
166             else:
167                 row[field] = str(hit)
168         resolved.append(row)
169     return resolved, issues
170
171
172 def resolve_existing_path(
173     raw_path: str,

```

```

174     field: str,
175     *,
176     manifest_path: str | os.PathLike[str],
177     repo_root_path: str | os.PathLike[str],
178     extra_roots: Iterable[str | os.PathLike[str]] = (),
179 ) -> Path | None:
180     """Resolve one manifest path field to an existing file, if possible."""
181
182     for candidate in _candidate_paths(
183         raw_path,
184         field,
185         Path(manifest_path),
186         Path(repo_root_path),
187         [Path(p) for p in extra_roots],
188     ):
189         if candidate.is_file():
190             return candidate.resolve()
191     return None
192
193
194 def _env_roots() -> list[Path]:
195     raw = os.environ.get(SEARCH_ROOTS_ENV, "")
196     if not raw:
197         return []
198     return [Path(p) for p in raw.split(os.pathsep) if p.strip()]
199
200
201 def _basename(raw_path: str) -> str:
202     # PureWindowsPath handles stale Windows paths even when tests run on POSIX.
203     win_name = PureWindowsPath(raw_path).name
204     return win_name or Path(raw_path).name
205
206
207 def _candidate_paths(
208     raw_path: str,
209     field: str,
210     manifest_path: Path,
211     root: Path,
212     extra_roots: Sequence[Path],
213 ) -> list[Path]:
214     basename = _basename(raw_path)
215     raw = Path(os.path.expandvars(os.path.expanduser(raw_path)))
216     manifest_dir = manifest_path.resolve().parent
217     candidates: list[Path] = [raw]
218
219     if not raw.is_absolute():
220         candidates.extend([manifest_dir / raw, root / raw])
221
222     for directory in _candidate_dirs(field, manifest_dir, root, extra_roots):
223         candidates.append(directory / basename)
224     return _dedup_paths(candidates)
225
226
227 def _candidate_dirs(
228     field: str, manifest_dir: Path, root: Path, extra_roots: Sequence[Path]
229 ) -> list[Path]:
230     field_dir = _FIELD_DIR.get(field)
231     dirs: list[Path] = [manifest_dir]
232     if field_dir:
233         dirs.extend(
234             [
235                 root.parent / "Annotation Setup" / field_dir,
236                 root.parent / "Annotation Setup" / "Collect" / field_dir,
237                 root / "output",
238             ]

```

```

239         )
240         dirs.extend(root_dirs for r in extra_roots for root_dirs in _expand_root(r, field))
241         return _dedup_paths(dirs)
242
243
244     def _expand_root(root: Path, field: str) -> list[Path]:
245         field_dir = _FIELD_DIR.get(field)
246         dirs = [root, root / "output"]
247         if field_dir:
248             dirs.extend(
249                 [
250                     root / field_dir,
251                     root / "Annotation Setup" / field_dir,
252                     root / "Annotation Setup" / "Collect" / field_dir,
253                 ]
254             )
255         return dirs
256
257
258     def _dedup_paths(paths: Iterable[Path]) -> list[Path]:
259         out: list[Path] = []
260         seen: set[str] = set()
261         for path in paths:
262             try:
263                 key = os.path.normcase(os.path.abspath(os.fspath(path)))
264             except OSError:
265                 key = os.fspath(path)
266             if key in seen:
267                 continue
268             seen.add(key)
269             out.append(path)
270         return out
271
272
273     def write_manifest(path: str | os.PathLike[str], specs: Sequence[dict[str, Any]]) ->
None:
274         """Write a resolved manifest JSON atomically enough for local tooling."""
275
276         out = Path(path)
277         out.parent.mkdir(parents=True, exist_ok=True)
278         tmp = out.with_name(out.name + ".tmp")
279         with tmp.open("w", encoding="utf-8") as f:
280             json.dump(list(specs), f, indent=2)
281             f.write("\n")
282         os.replace(tmp, out)
283
284
285     def build_parser() -> argparse.ArgumentParser:
286         p = argparse.ArgumentParser(
287             description="Check/rebase the local golden corpus manifest paths."
288         )
289         p.add_argument("--manifest", default=DEFAULT_MANIFEST)
290         p.add_argument(
291             "--repo-root",
292             default=None,
293             help="repo root used for default sibling output/Annotation Setup lookups",
294         )
295         p.add_argument(
296             "--root",
297             action="append",
298             default=[],
299             help=(
300                 "additional search root; repeatable. Also accepts roots via "
301                 f"{SEARCH_ROOTS_ENV} separated by os.pathsep."
302             ),

```

```

303     )
304     p.add_argument(
305         "--require",
306         action="append",
307         choices=sorted(_FIELD_DIR),
308         default=None,
309         help="path field to require; repeatable. Defaults to trajectory and labels.",
310     )
311     p.add_argument(
312         "--min-count",
313         type=int,
314         default=None,
315         help="fail unless at least this many manifest rows are present",
316     )
317     p.add_argument(
318         "--write",
319         default=None,
320         help="optional path to write the resolved local manifest JSON",
321     )
322     return p
323
324
325 def main(argv: Sequence[str] | None = None) -> int:
326     args = build_parser().parse_args(argv)
327     required = tuple(args.require or DEFAULT_REQUIRED_FIELDS)
328     try:
329         original = load_manifest(args.manifest)
330         resolved, issues = resolve_manifest_specs(
331             original,
332             args.manifest,
333             repo_root_path=args.repo_root,
334             extra_roots=args.root,
335             required_fields=required,
336         )
337     except (OSError, ValueError, json.JSONDecodeError) as e:
338         print(f"ERROR: {e}", file=sys.stderr)
339         return 2
340
341     if args.min_count is not None and len(resolved) < args.min_count:
342         issues.append(
343             ManifestIssue(
344                 "manifest",
345                 "rows",
346                 str(len(resolved)),
347                 f"expected at least {args.min_count} row(s)",
348             )
349         )
350
351     if issues:
352         print(
353             f"ERROR: golden manifest path check failed ({len(issues)} issue(s)):",
354             file=sys.stderr,
355         )
356         for issue in issues:
357             print(f" - {issue.message()}", file=sys.stderr)
358         return 2
359
360     rebased = sum(
361         1
362         for before, after in zip(original, resolved)
363         for field in required
364         if str(before.get(field, "")) != str(after.get(field, ""))
365     )
366     print(
367         f"OK: {args.manifest} has {len(resolved)} video(s); "

```

```

368         f"{len(resolved) * len(required)} required path(s) present; "
369         f"{rebased} path(s) rebased."
370     )
371     if args.write:
372         write_manifest(args.write, resolved)
373         print(f"wrote resolved manifest: {args.write}")
374     return 0
375
376
377     if __name__ == "__main__":
378         raise SystemExit(main())
379

```

16. assistant (2026-07-03T17:33:49.843Z)

Now I understand the behavior change. Let me analyze the critical differences:

```

**OLD behavior (test_served_gate_a_regression.py):**
- Loads JSON manually
- Filters per-spec: keeps only specs where BOTH trajectory AND labels files exist
- Returns filtered list
- If >= 6 filtered specs exist, use them; else return []

**NEW behavior (test_served_gate_a_regression.py):**
- Calls `load_resolved_manifest(MANIFEST)` with `strict=True` (default)
- If ANY spec has a missing trajectory OR labels file, raises ManifestPathError
- Catches exception and returns []
- If >= 6 specs exist, use them; else return []

```

The key issue: The old code would skip individual specs with missing files and still run tests on the remaining ones. The new code will fail the entire load if ANY file is missing.

Now let me check the other consumers:

17. user (2026-07-03T17:33:51.394Z)

```

diff --git a/backend/eval/calibrate_local.py b/backend/eval/calibrate_local.py
index 687f790..4734f38 100644
--- a/backend/eval/calibrate_local.py
+++ b/backend/eval/calibrate_local.py
@@ -24,7 +24,6 @@ where videos.json = [{"name", "trajectory", "fps", "frame_width", "labels"}, ...]
     from __future__ import annotations

     import argparse
-    import json
     from typing import Callable, Dict, List, Tuple

     from backend.eval.calibrate_wasb import load_golden_gts
@@ -34,6 +33,7 @@ from backend.eval.calibration import (
     leave_one_video_out,
     select_best,
 )
+from backend.eval.golden_manifest import load_resolved_manifest
     from backend.pipeline.detectors import rally_gate
     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

@@ -164,8 +164,7 @@ def main() -> None:
     ap.add_argument("--iou", type=float, default=0.5)
     ap.add_argument("--metric", default="f1", choices=["f1", "precision", "recall"])
     args = ap.parse_args()
-    with open(args.manifest, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(args.manifest)
     run(specs, iou=args.iou, metric=args.metric)

```

```

diff --git a/backend/eval/distill_local.py b/backend/eval/distill_local.py
index 52ac85a..a05b3ff 100644
--- a/backend/eval/distill_local.py
+++ b/backend/eval/distill_local.py
@@ -24,13 +24,13 @@ Run:
     from __future__ import annotations

     import argparse
-import json
+from typing import Dict, List, Optional, Tuple

+from backend.eval import windowing as _windowing
+from backend.eval.calibrate_wasb import load_golden_gts
+from backend.eval.classifier import LogisticRegression
+from backend.eval.gemini_refine import merge_overlaps
+from backend.eval.golden_manifest import load_resolved_manifest
+from backend.eval.metrics import score
+from backend.eval.training import label_candidates
+from backend.pipeline.detectors import rally_gate
@@ -218,8 +218,7 @@ def main() -> None:
    )
    ap.add_argument("--model-out", default="output/local_rally_model.json")
    args = ap.parse_args()
-    with open(args.manifest, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(args.manifest)
    run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)

diff --git a/backend/eval/fusion_golden.py b/backend/eval/fusion_golden.py
index dd3f6a1..aee6d7e 100644
--- a/backend/eval/fusion_golden.py
+++ b/backend/eval/fusion_golden.py
@@ -25,6 +25,7 @@ from typing import Any, Dict, List, Optional

+from backend.eval import distill_local
+from backend.eval.calibrate_wasb import load_golden_gts
+from backend.eval.golden_manifest import load_resolved_manifest
+from backend.eval.training import label_candidates
+from backend.pipeline.detectors import fusion_features as ff
+from backend.pipeline.detectors import rally_gate
@@ -160,8 +161,7 @@ def run(
    fresh: bool = False,
    smallest_first: bool = False,
) -> List[str]:
-    with open(manifest_path, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(manifest_path)
    if smallest_first:
        # Process small videos first for quick feedback (the slow full match lands last);
        # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).

diff --git a/backend/eval/rally_seg_eval.py b/backend/eval/rally_seg_eval.py
index 0acca2d..3846ce0 100644
--- a/backend/eval/rally_seg_eval.py
+++ b/backend/eval/rally_seg_eval.py
@@ -27,6 +27,7 @@ import os
+from typing import Any, Dict, List, Optional, Tuple

+from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
+from backend.eval.golden_manifest import load_resolved_manifest
+from backend.eval.metrics import Interval
+from backend.eval.windowing import PRESETS, SERVED, WindowingPreset

@@ -185,8 +186,7 @@ def load_golden(

```

```

) -> List[Dict[str, Any]]:
    """Load each golden video's trajectory path + fps + frame_width (manifest) and GT rally
    intervals (`<name>_golden_features.json` ``gts``). Videos with no GTs are kept but
    flagged."""
    with open(manifest_path) as fh:
        manifest = json.load(fh)
+   manifest = load_resolved_manifest(manifest_path, required_fields=("trajectory",))
    out: List[Dict[str, Any]] = []
    for v in manifest:
        feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
diff --git a/backend/eval/rally_seq_proto.py b/backend/eval/rally_seq_proto.py
index 2ea9483..b017f94 100644
--- a/backend/eval/rally_seq_proto.py
+++ b/backend/eval/rally_seq_proto.py
@@ -29,7 +29,6 @@ Run:
     from __future__ import annotations

     import argparse
-    import json
     from typing import Dict, List, Tuple

     import numpy as np
@@ -38,6 +37,7 @@ from scipy.stats import rankdata

     from backend.eval.calibrate_wasb import load_golden_gts
     from backend.eval.classifier import LogisticRegression
+    from backend.eval.golden_manifest import load_resolved_manifest
     from backend.eval.metrics import score

     # Feature extraction is single-sourced in backend/features (ADR-008): training and
@@ -196,8 +196,7 @@ def main() -> None:
     help="JSON: [{name,trajectory,fps,frame_width,labels}, ...]",
     )
     args = ap.parse_args()
-    with open(args.manifest, encoding="utf-8") as f:
-        run(json.load(f))
+    run(load_resolved_manifest(args.manifest))

     if __name__ == "__main__":
diff --git a/backend/eval/serve_contrast.py b/backend/eval/serve_contrast.py
index 328857b..0f1b1f4 100644
--- a/backend/eval/serve_contrast.py
+++ b/backend/eval/serve_contrast.py
@@ -32,6 +32,7 @@ from typing import Any, Dict, List, Sequence, Tuple
     from backend.eval import windowing as W
     from backend.eval.calibrate_wasb import load_golden_gts
     from backend.eval.gemini_refine import merge_overlaps
+    from backend.eval.golden_manifest import load_resolved_manifest
     from backend.eval.metrics import score
     from backend.eval.promotion import promote_if_served_safe
     from backend.pipeline.detectors import rally_gate
@@ -194,8 +195,7 @@ def main() -> None:
     )
     args = ap.parse_args()

-    with open(args.manifest, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(args.manifest)
     result = run_contrast(
         specs,
         iou=args.iou,
diff --git a/backend/eval/served_gate_a.py b/backend/eval/served_gate_a.py
index 994b9c1..c2151fb 100644
--- a/backend/eval/served_gate_a.py

```

```

+++ b/backend/eval/served_gate_a.py
@@ -23,13 +23,13 @@ Run (owner box, golden trajectories present):
    from __future__ import annotations

    import argparse
-import json
    from dataclasses import dataclass
    from typing import Any, Dict, List, Optional, Tuple

    from backend.config.models import FusionConfig, IndexingConfig, WasbIndexConfig
    from backend.eval import segmentation_metrics as _segm
    from backend.eval.calibrate_wasb import load_golden_gts
+from backend.eval.golden_manifest import load_resolved_manifest
    from backend.eval.metrics import Interval, Score, score
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
    from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
@@ -196,8 +196,7 @@ def format_table(
    def run(
        manifest_path: str, iou: float = 0.5
    ) -> Tuple[List[ServedVideoResult], Dict[str, Any]]:
-    with open(manifest_path, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(manifest_path)
        results = evaluate_manifest(specs, iou=iou)
        return results, aggregate(results)

diff --git a/training/gen0/harness.py b/training/gen0/harness.py
index 6eec29f..433d2cb 100644
--- a/training/gen0/harness.py
+++ b/training/gen0/harness.py
@@ -36,6 +36,7 @@ from typing import Dict, List, Optional
    import numpy as np

    from backend.eval.classifier import LogisticRegression
+from backend.eval.golden_manifest import load_resolved_manifest
    from backend.eval.gt_loader import load_gt_intervals
    from backend.eval.rally_seq_proto import (
        _seg_f1,
@@ -217,8 +218,7 @@ def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
    def run(
        manifest_path: str, out_dir: str, version: str, floor_path: Optional[str] = None
    ) -> dict:
-    with open(manifest_path, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(manifest_path)
        floor = None
        if floor_path:
            with open(floor_path, encoding="utf-8") as f:

```

18. assistant (2026-07-03T17:33:53.582Z)

Good. Now let me check if there are any grep calls using json in these files to verify the `import json` removals don't break anything:

19. user (2026-07-03T17:33:55.665Z)

```

=== backend/eval/calibrate_local.py ===
no json usage
=== backend/eval/distill_local.py ===
no json usage
=== backend/eval/fusion_golden.py ===
    json.dump(data, f, indent=2)
=== backend/eval/rally_seg_eval.py ===
        (float(s), float(e)) for s, e in (json.load(fh).get("gts") or [])
    json.dump(result, fh, indent=2)

```

```

=== backend/eval/rally_seq_proto.py ===
no json usage
=== backend/eval/serve_contrast.py ===
    json.dump(result, f, indent=2)
=== backend/eval/served_gate_a.py ===
no json usage
=== training/gen0/harness.py ===
    floor = json.load(f)
    json.dump(payload, f, indent=2)
    json.dump(manifest, f, indent=2)
=== tests/test_served_gate_a_regression.py ===
no json usage

```

20. assistant (2026-07-03T17:33:57.721Z)

Found issues! Some files still use `json` but removed the import. Let me check those files:

21. user (2026-07-03T17:33:58.315Z)

```

1      """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3      Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4      candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5      EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6      writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7      CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8      measured by the shipped experiment harness; person features enter only as feature
columns the
9      learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11     Run (on the GPU box):
12     python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14     Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15     `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16     """
17
18     from __future__ import annotations
19
20     import argparse
21     import json
22     import os
23     import time
24     from typing import Any, Dict, List, Optional
25
26     from backend.eval import distill_local
27     from backend.eval.calibrate_wasb import load_golden_gts
28     from backend.eval.golden_manifest import load_resolved_manifest
29     from backend.eval.training import label_candidates
30     from backend.pipeline.detectors import fusion_features as ff
31     from backend.pipeline.detectors import rally_gate
32     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
33     from backend.utils.run_telemetry import RunTelemetry
34
35     # The 5 fps/resolution-invariant shuttle features distill_local already computes.
36     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
37     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
38     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)

```

```

39
40
41     def _derive_video_path(spec: Dict[str, Any]) -> str:
42         """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
43         golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
44         convention)."""
45         if spec.get("video_path"):
46             return str(spec["video_path"])
47         labels = str(spec["labels"])
48         if labels.endswith(".rallies.csv"):
49             return labels[: -len(".rallies.csv")] + ".MP4"
50         raise ValueError(
51             f"{spec.get('name')}: no video_path and labels {labels!r} isn't *.rallies.csv"

```

22. assistant (2026-07-03T17:34:00.489Z)

Good, fusion_golden.py still imports json. Let me check rally_seg_eval.py more carefully:

23. user (2026-07-03T17:34:01.516Z)

```

diff --git a/backend/eval/rally_seg_eval.py b/backend/eval/rally_seg_eval.py
index 0acca2d..3846ce0 100644
--- a/backend/eval/rally_seg_eval.py
+++ b/backend/eval/rally_seg_eval.py
@@ -27,6 +27,7 @@ import os
 from typing import Any, Dict, List, Optional, Tuple

 from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
+from backend.eval.golden_manifest import load_resolved_manifest
 from backend.eval.metrics import Interval
 from backend.eval.windowing import PRESETS, SERVED, WindowingPreset

@@ -185,8 +186,7 @@ def load_golden(
 ) -> List[Dict[str, Any]]:
     """Load each golden video's trajectory path + fps + frame_width (manifest) and GT rally
     intervals (`<name>_golden_features.json` `gts`). Videos with no GTs are kept but
     flagged."""
-    with open(manifest_path) as fh:
-        manifest = json.load(fh)
+    manifest = load_resolved_manifest(manifest_path, required_fields=("trajectory",))
     out: List[Dict[str, Any]] = []
     for v in manifest:
         feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")

```

24. assistant (2026-07-03T17:34:03.205Z)

Let me check the full rally_seg_eval.py for imports:

25. user (2026-07-03T17:34:03.939Z)

```

1     """One-command rally-segmentation eval ? score a windowing against the golden labels
2     with the
3     over-segmentation-sensitive metric set, honestly.
4     This is the harness the rally-quality project (`docs/RALLY_QUALITY_RESEARCH.md` $4.5 /
5     $7)
6     measures every tier against. It closes the loop the over-merge finding exposed: plain
7     temporal
8     IoU-F1 is blind to merged mega-windows, so we report it **alongside** segmental F1@k,
9     the
10    segment-count ratio, and the explicit `merge_split` breakdown
11    (`segmentation_metrics`).
12
13    For each golden video it: parses the cached WASB/TrackNet trajectory CSV ? builds

```

```

candidate
10     windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the
SERVED
11     operating point and any future windowing the same way) ? scores the windows-as-rallies
against
12     the ground-truth rally intervals. Aggregates per-video held-out F1 with a bootstrap CI,
and can
13     diff two windowings with a paired sign-test + CI (the per-tier ``delta``).
14
15     GROUND TRUTH: the golden manifest (``output/human_lovo_manifest.json``) gives each
video's
16     trajectory path + fps + frame_width; the GT rally intervals come from
17     ``output/<name>_golden_features.json`` (the ``gts`` key) so this needs no extra label
files.
18
19     Pure offline (CPU): trajectory parsing + windowing + interval scoring; no GPU/Gemini.
20     """
21
22     from __future__ import annotations
23
24     import argparse
25     import json
26     import os
27     from typing import Any, Dict, List, Optional, Tuple
28
29     from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
30     from backend.eval.golden_manifest import load_resolved_manifest
31     from backend.eval.metrics import Interval
32     from backend.eval.windowing import PRESETS, SERVED, WindowingPreset
33
34     DEFAULT_MANIFEST = os.path.join("output", "human_lovo_manifest.json")
35     DEFAULT_FEATURES_DIR = "output"
36     DEFAULT_IOU = 0.5
37
38     #: Long-rally lens threshold (s). A rally is "long" (an eye-catching highlight) when its
duration
39     #: (end ? start) exceeds this. The local detector over-fires, and its false positives
are mostly
40     #: SHORT (motion blips, background-court fragments on multicourt footage); filtering to
long rallies

```

26. assistant (2026-07-03T17:34:05.906Z)

OK, rally_seg_eval.py still imports json (line 25). Now let me check the test changes:

27. user (2026-07-03T17:34:07.139Z)

```

diff --git a/tests/test_fusion_compare.py b/tests/test_fusion_compare.py
index d3477c8..f042eea 100644
--- a/tests/test_fusion_compare.py
+++ b/tests/test_fusion_compare.py
@@ -203,6 +203,8 @@ def test_fusion_golden_run_is_resumable(tmp_path, monkeypatch):

     out_dir = str(tmp_path / "out")
     os.makedirs(out_dir)
+    for name in ("a.csv", "b.csv", "a.rallies.csv", "b.rallies.csv"):
+        (tmp_path / name).write_text("x\n", encoding="utf-8")
     manifest = [
         {
             "name": "alpha",
@@ -267,6 +269,8 @@ def test_fusion_golden_smallest_first(tmp_path, monkeypatch):
     os.makedirs(out_dir)
     (tmp_path / "big.csv").write_text("x" * 2000)
     (tmp_path / "small.csv").write_text("x" * 20)
+    (tmp_path / "b.rallies.csv").write_text("x\n", encoding="utf-8")

```

```

+ (tmp_path / "s.rallies.csv").write_text("x\n", encoding="utf-8")
manifest = [
    {
        "name": "big",
diff --git a/tests/test_fusion_golden_telemetry.py b/tests/test_fusion_golden_telemetry.py
index 1fab90..bdefc88 100644
--- a/tests/test_fusion_golden_telemetry.py
+++ b/tests/test_fusion_golden_telemetry.py
@@ -45,6 +45,9 @@ def _patch_telemetry(monkeypatch, gpu_fn=_fake_gpu, sys_fn=_fake_sys):

def _write_manifest(tmp_path, names):
+ for n in names:
+ (tmp_path / f"{n}.csv").write_text("x\n", encoding="utf-8")
+ (tmp_path / f"{n}.rallies.csv").write_text("x\n", encoding="utf-8")
manifest = [
    {
        "name": n,

```

28. assistant (2026-07-03T17:34:12.775Z)

Excellent! These test files now CREATE the trajectory and labels files to make the strict loader work. Let me check what the original tests were doing:

29. user (2026-07-03T17:34:13.719Z)

```

def test_fusion_golden_run_is_resumable(tmp_path, monkeypatch):
    """run() skips a video whose JSON already exists (resume a long run); --fresh redoes all."""
    from backend.eval import fusion_golden

    out_dir = str(tmp_path / "out")
    os.makedirs(out_dir)
    manifest = [
        {
            "name": "alpha",
            "trajectory": "a.csv",
            "fps": 30.0,
            "frame_width": 1920.0,
            "labels": "a.rallies.csv",
        },
        {
            "name": "beta",
            "trajectory": "b.csv",
            "fps": 30.0,
            "frame_width": 1920.0,
            "labels": "b.rallies.csv",
        },
    ]
    manifest_path = str(tmp_path / "m.json")
    with open(manifest_path, "w", encoding="utf-8") as f:
        json.dump(manifest, f)
    # a prior run already produced alpha's JSON
    with open(
        os.path.join(out_dir, "alpha_golden_features.json"), "w", encoding="utf-8"
    ) as f:
        json.dump(
            {

```

30. assistant (2026-07-03T17:34:16.679Z)

Perfect! The old test did NOT create the trajectory/labels files?it only created the manifest JSON. The new test ADDS file creation before calling fusion_golden.run(), which needs files to exist because of the strict loader.

Now let me check if there are any callers that might break. Let me search for nightly_regression.py:

31. user (2026-07-03T17:36:00.859Z)

/c/\$Recycle.Bin/S-1-5-21-767216898-525181750-4010716354-1001/\$RG4UT35/badminton-highlight-indexer/scripts/nightly_regression.py

32. assistant (2026-07-03T17:36:02.715Z)

Let me search in the project directory:

33. user (2026-07-03T17:36:04.114Z)

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/scripts/nightly_regression.py

34. user (2026-07-03T17:36:06.356Z)

```
1      """Nightly rally-quality regression tripwire.
2
3      Re-scores the **golden corpus** under HEAD via :mod:`backend.eval.rally_seg_eval` (CPU,
4      seconds ? the WASB GPU trajectories are CACHED, only the windowing/scorer changes) and
5      diffs
6      the headline metrics against a FROZEN baseline
7      (`eval_baselines/nightly_baseline.json`).
8      Writes a dated report and exits non-zero on any regression beyond tolerance OR any per-
9      video
10     over-segmentation-guard regression.
11
12     This is the owner's dual-eval-set discipline made automatic (`memory/eval-dual-set-
13     regression`
14     + `docs/R2_EVAL_METRIC_DESIGN.md` $4.5): the growing GOLDEN-finalized set is re-run
15     every
16     night so a *late* change cannot silently lose an *earlier* win. It is **default-
17     ADDITIVE** ? it
18     does NOT change the promotion gate. It is a *tripwire*, not a re-training run: no GPU,
19     no Gemini,
20     no WASB re-extraction (those are cached and unchanged night to night).
21
22     Three tracked configs (all re-scored every night; a clean A/B/C on the same SERVED base
23     knobs):
24
25     * **DEFAULT** ? the SERVED production windowing, i.e. what users get today. Catches a
26     regression
27     to shipped behaviour. **NOTE (post-#204, 2026-06-18):** the R1 milestone is now
28     flipped ON, so
29     SERVED *is* cap=6 + R4 ? "default" now equals "milestone".
30     * **MILESTONE** ? the explicit R3+R4 config (cap=6 s + rally-END inactivity trim;
31     `docs/QUALITY_ITERATIONS.md` R3/R4). Now == DEFAULT (production shipped it); kept
32     explicit so
33     the nightly still names the milestone knobs and stays meaningful if production later
34     diverges.
35     * **BASELINE_OFF** ? the pre-R1-flip all-OFF windowing (every quality knob forced OFF).
36     The
37     permanent historical floor, so each report keeps showing the delta the milestone buys
38     (a launched
39     win must keep standing its ground, not be taken at launch-time value).
40
41     Ground truth + trajectories are LOCAL (the manifest points at owned, minors-free
42     trajectory CSVs
43     outside the repo + `output/<name>_golden_features.json`); the corpus is NOT committed.
44     So the
45     baseline is tied to the owner's local golden corpus and is regenerated with `--update-
46     baseline`
```

```

30     whenever the corpus grows or an intended improvement lands. (A cloud agent does not
31     suffice ? it
32     has the repo but not the cached trajectories; hence the local Windows Scheduled Task
33     wiring in
34     ``scripts/nightly_regression.ps1``.)
35     Usage
36     -----
37     Snapshot the baseline from current HEAD (after an intended improvement or a corpus
38     change):
39     python scripts/nightly_regression.py --update-baseline
40     Nightly check (exit 1 on regression; writes ``output/nightly_regression/<date>.md`` +
41     ``.json``):
42     python scripts/nightly_regression.py
43
44     Exit codes
45     -----
46     * ``0`` ? all tracked configs clean (no regression), corpus matches the baseline.
47     * ``1`` ? at least one regression (an aggregate metric dropped beyond tolerance, or a
48     per-video
49     over-seg guard increased). Takes precedence over a stale baseline.
50     * ``2`` ? operational error (manifest/corpus missing, nothing scored).
51     * ``3`` ? no baseline to compare against (snapshot one with ``--update-baseline``).
52     * ``4`` ? baseline STALE (the corpus set or a config definition changed): a refresh is
53     needed,
54     but no regression was found on the comparable intersection.
55     ""
56
57     from __future__ import annotations
58
59     import argparse
60     import datetime as _dt
61     import json
62     import os
63     import sys
64     from dataclasses import dataclass, field, replace
65     from typing import Any, Dict, List, Optional, Tuple
66
67     # ----- #
68     # Paths (resolved relative to the repo root, so the script is cwd-robust).
69     # ----- #
70     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
71     if REPO_ROOT not in sys.path:
72         sys.path.insert(
73             0, REPO_ROOT
74         ) # importable as `python scripts/nightly_regression.py` from anywhere
75     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "nightly_baseline.json")
76     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_regression")
77     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "output", "human_lovo_manifest.json")
78     DEFAULT_FEATURES_DIR = os.path.join(REPO_ROOT, "output")
79     DEFAULT_IOU = 0.5
80
81     # ----- #
82     # What we gate on (a tripwire ? focused, not every metric).
83     # * HIGHER-better aggregates regress when they DROP beyond ``metric_tol``.
84     # * merge_rate (lower-better) regresses when it RISES beyond ``rate_tol``.
85     # * per-video split_count / merge_count must NOT increase beyond ``over_seg_tol``
86     # (the R2 §2.3.3 guard: "may not increase EITHER on ANY video").
87     # Boundary-error medians + seg_ratio are outlier-dominated diagnostics ? REPORTED, not
88     gated.
89     # ----- #
90     GATED_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")

```

```

88     GATED_LOWER: Tuple[str, ...] = ("merge_rate",)
89     PER_VIDEO_GUARD: Tuple[str, ...] = ("split_count", "merge_count")
90
91     #: Aggregate metrics snapshotted into the baseline (the gated subset + report-only
context).
92     AGG_SNAPSHOT: Tuple[str, ...] = (
93         "tol_f1",
94         "tol_precision",
95         "tol_recall",
96         "f1@0.5",
97         "f1@0.25",
98         "precision",
99         "recall",
100        "merge_rate",

```

35. assistant (2026-07-03T17:36:09.181Z)

Let me check if it calls rally_seg_eval.load_golden:

36. user (2026-07-03T17:36:10.475Z)

```

3:Re-scores the **golden corpus** under HEAD via :mod:`backend.eval.rally_seg_eval` (CPU,
216:     """Compact, comparable summary of one :func:`rally_seg_eval.evaluate` result.
629:     from backend.eval import rally_seg_eval
631:     golden = rally_seg_eval.load_golden(manifest, features_dir)
644:     result = rally_seg_eval.evaluate(

```

37. assistant (2026-07-03T17:36:12.656Z)

Good! Let me look at that call:

38. user (2026-07-03T17:36:12.810Z)

```

625
626     ``snapshot`` is the serialisable structure stored as / compared to the baseline.
627     ``skipped``
628     lists manifest videos NOT scored (missing trajectory or empty GT) per config ?
surfaced in
629     the report so the corpus can't silently shrink."""
630     from backend.eval import rally_seg_eval
631
632     golden = rally_seg_eval.load_golden(manifest, features_dir)
633     manifest_names = [v["name"] for v in golden]
634     if not golden:
635         raise RuntimeError(f"no golden videos loaded from {manifest}")
636
637     snapshot: Dict[str, Any] = {
638         "manifest": _display_path(manifest),
639         "iou": iou,
640         "configs": {},
641     }
642     skipped: Dict[str, List[str]] = {}
643     scored_any = False
644     for cfg in configs:
645         result = rally_seg_eval.evaluate(
646             golden, cfg.preset, iou=iou, min_crossings=cfg.min_crossings
647         )
648         summary = summarize_run(result)
649         summary["note"] = cfg.note
650         snapshot["configs"][cfg.name] = summary
651         scored = set(summary["per_video"])
652         skipped[cfg.name] = [n for n in manifest_names if n not in scored]
653         scored_any = scored_any or bool(scored)

```

```

654         raise RuntimeError(
655             f"scored 0 videos for every config ? are the trajectory CSVs in {manifest}
present "
656             f"and do the {features_dir}/<name>_golden_features.json files have 'gts'?"
657         )
658     return snapshot, skipped
659
660
661 def load_baseline(path: str) -> Optional[Dict[str, Any]]:
662     if not os.path.exists(path):
663         return None
664     with open(path, encoding="utf-8") as fh:
665         return json.load(fh)
666
667
668 def save_baseline(path: str, snapshot: Dict[str, Any], tols: Tolerances) -> None:
669     out = dict(snapshot)
670     out["generated_at"] = _dt.date.today().isoformat()
671     out["git_commit"] = _git_commit()
672     out["tolerances"] = tols.to_dict()
673     out["_doc"] = (
674         "Frozen nightly rally-quality regression baseline. Regenerate with "
675         "`python scripts/nightly_regression.py --update-baseline` after an intended "
676         "improvement or a corpus change. Tied to the LOCAL golden corpus (not
committed)."
677     )
678     os.makedirs(os.path.dirname(path), exist_ok=True)
679     tmp = path + ".tmp"
680     with open(tmp, "w", encoding="utf-8") as fh:
681         json.dump(out, fh, indent=2, sort_keys=True)
682         fh.write("\n")
683     os.replace(tmp, path)
684
685
686 def build_parser() -> argparse.ArgumentParser:
687     p = argparse.ArgumentParser(
688         description="Nightly rally-quality regression tripwire (offline, CPU)."
689     )
690     p.add_argument("--manifest", default=DEFAULT_MANIFEST, help="golden manifest JSON")
691     p.add_argument(
692         "--features-dir",
693         default=DEFAULT_FEATURES_DIR,
694         help="dir holding <name>_golden_features.json",
695     )
696     p.add_argument(
697         "--baseline", default=DEFAULT_BASELINE, help="frozen baseline JSON path"
698     )
699     p.add_argument(
700         "--report-dir",
701         default=DEFAULT_REPORT_DIR,
702         help="where dated reports are written",
703     )
704     p.add_argument("--iou", type=float, default=DEFAULT_IOU)
705     p.add_argument(
706         "--metric-tol",
707         type=float,
708         default=DEFAULT_METRIC_TOL,
709         help="F1-like aggregate drop that counts as a regression",
710     )
711     p.add_argument(
712         "--rate-tol",
713         type=float,
714         default=DEFAULT_RATE_TOL,
715         help="merge_rate rise that counts as a regression",
716     )

```

```

717     p.add_argument(
718         "--over-seg-tol",
719         type=int,
720         default=DEFAULT_OVER_SEG_TOL,
721         help="per-video split/merge-count increase allowed before it's a regression",
722     )
723     p.add_argument(
724         "--update-baseline",
725         action="store_true",
726         help="snapshot the current numbers as the new baseline instead of checking",
727     )
728     p.add_argument(
729         "--date", default=None, help="override the report date (YYYY-MM-DD; for tests)"
730     )
731     p.add_argument(
732         "--no-report", action="store_true", help="skip writing the dated report file"
733     )
734     return p
735
736
737 def main(argv: Optional[List[str]] = None) -> int:
738     import logging
739
740     logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
741     log = logging.getLogger("nightly_regression")
742     args = build_parser().parse_args(argv)
743     tols = Tolerances(
744         metric_tol=args.metric_tol,
745         rate_tol=args.rate_tol,
746         over_seg_tol=args.over_seg_tol,
747     )
748     date = args.date or _dt.date.today().isoformat()
749
750     if not os.path.exists(args.manifest):
751         log.error(
752             "manifest not found: %s ? the golden corpus must be present locally.",
753             args.manifest,
754         )
755         return 2
756
757     try:
758         snapshot, skipped = run_corpus(
759             tracked_configs(), args.manifest, args.features_dir, args.iou
760         )
761     except RuntimeError as e:
762         log.error("%s", e)
763         return 2
764     snapshot["git_commit"] = _git_commit()
765     snapshot["generated_at"] = date
766
767     if args.update_baseline:
768         save_baseline(args.baseline, snapshot, tols)
769         log.info("baseline snapshot written to %s", args.baseline)
770         for cfg, s in snapshot["configs"].items():
771             agg = s["aggregate"]
772             log.info(
773                 " %s: n=%s tolF1=%.3f f1@0.5=%.3f merge_rate=%.3f",
774                 cfg,
775                 s["n"],
776                 agg.get("tol_f1") or 0.0,
777                 agg.get("f1@0.5") or 0.0,
778                 agg.get("merge_rate") or 0.0,
779             )
780     return 0
781

```

```

782     baseline = load_baseline(args.baseline)
783     if baseline is None:
784         log.error(
785             "no baseline at %s ? snapshot one with --update-baseline (then commit it).",
786             args.baseline,
787         )
788         return 3
789
790     result = compare_all(baseline, snapshot, tols, skipped)
791     report = format_report(baseline, snapshot, result, tols, date)
792
793     if not args.no_report:
794         os.makedirs(args.report_dir, exist_ok=True)
795         md_path = os.path.join(args.report_dir, f"{date}.md")
796         json_path = os.path.join(args.report_dir, f"{date}.json")
797         with open(md_path, "w", encoding="utf-8") as fh:
798             fh.write(report + "\n")
799         with open(json_path, "w", encoding="utf-8") as fh:
800             json.dump(
801                 {
802                     "date": date,
803                     "status": result.overall_status(),
804                     "exit_code": result.exit_code(),
805                     "current": snapshot,
806                     "regressions": [
807                         f.message() for c in result.comparisons for f in c.regressions
808                     ],
809                 },
810                 fh,
811                 indent=2,
812                 sort_keys=True,
813             )
814         log.info("report written to %s", md_path)
815
816     print(report)
817     code = result.exit_code()
818     if code == 1:
819         log.error(
820             "NIGHTLY REGRESSION DETECTED (%d) ? see %s",
821             sum(len(c.regressions) for c in result.comparisons),
822             args.report_dir,
823         )
824     elif code == 4:
825         log.warning(
826             "baseline STALE ? corpus/config changed; refresh with --update-baseline"
827         )
828     else:
829         log.info("nightly regression check PASSED")
830     return code
831
832
833 if __name__ == "__main__":
834     raise SystemExit(main())
835

```

39. assistant (2026-07-03T17:36:14.851Z)

Now let me look at rally_seg_eval.load_golden to see how it uses the manifest:

40. user (2026-07-03T17:36:15.617Z)

```

180
181 # ----- #
182 # Golden corpus I/O
183 # ----- #

```

```

184     def load_golden(
185         manifest_path: str = DEFAULT_MANIFEST, features_dir: str = DEFAULT_FEATURES_DIR
186     ) -> List[Dict[str, Any]]:
187         """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
rally
188         intervals (`<name>_golden_features.json` ``gts``). Videos with no GTs are kept but
flagged."""
189         manifest = load_resolved_manifest(manifest_path, required_fields=("trajectory",))
190         out: List[Dict[str, Any]] = []
191         for v in manifest:
192             feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
193             gts: List[Interval] = []
194             if os.path.exists(feat):
195                 with open(feat) as fh:
196                     gts = [
197                         (float(s), float(e)) for s, e in (json.load(fh).get("gts") or [])
198                     ]
199             entry: Dict[str, Any] = {
200                 "name": v["name"],
201                 "trajectory": v["trajectory"],
202                 "fps": float(v["fps"]),
203                 "frame_width": float(v["frame_width"]),
204                 "gts": gts,
205             }
206             # Optional prominent-court gate inputs (multicourt G2; default absent ? the gate
is a no-op):
207             # a normalized court_polygon + optional frame_height + court_height_pad from the
manifest.
208             for k in ("court_polygon", "frame_height", "court_height_pad"):
209                 if k in v:
210                     entry[k] = v[k]
211             out.append(entry)
212         return out
213
214
215     def windows_for(
216         video: Dict[str, Any], preset: WindowingPreset, min_crossings: int = 0
217     ) -> List[Interval]:
218         """Parse a video's trajectory, window it under ``preset`` ? predicted rally
intervals.
219
220         ``min_crossings`` > 0 applies the **net-crossings rally-state gate** (Gate-A,
221         ``rally_gate.gate_windows``): drop windows whose shuttle crosses the net fewer than
this many
222         times (a non-rally fragment). This is the cheapest rally-state cue (W2) ? CPU-only,
derived
223         from the trajectory + an auto-estimated net axis; it cleans up the over-split that
bounded
224         windowing (W1) leaves behind.
225
226         NEVER-EMPTY safeguard (mirrors ``FusionSegmenter._select_for_handoff``): if the gate
would
227         drop EVERY window of a video that had ?1, fall back to the ungated windows. The gate
is
228         strictly *pruning* ? it never zeroes out a non-empty video ? so W2-on-W1 is "do no
harm".
229         """
230         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
231
232         points = TrackNetRunner.parse_trajectory_csv(video["trajectory"])
233         wins = windowing.build_windows(points, video["fps"], video["frame_width"], preset)
234         ivs = windowing.windows_to_intervals(wins)
235         # Prominent-court WINDOW gate (multicourt G2, default-OFF): when the video carries a
normalized
236         # court_polygon, drop candidate rallies whose shuttle is MAJORITY outside the

```

```

prominent court ? a
237         # background/adjacent-court rally. Window-level (not point-level): a background
rally's central
238         # tail keeps a few in-court points, enough to hold a window, so point-clearing alone
is too leaky
239         # (measured on GX010142 rally #1). No polygon ? no-op. See
docs/PROMINENT_COURT_DETECTION.md.
240         poly = video.get("court_polygon")
241         if poly and ivs:
242             from backend.pipeline.detectors.court_gate import gate_intervals_to_court
243
244             fw = float(video["frame_width"])
245             fh = float(video.get("frame_height") or fw * 9.0 / 16.0)
246             ivs = gate_intervals_to_court(
247                 ivs,
248                 points,
249                 poly,
250                 float(video["fps"]),
251                 fw,
252                 fh,
253                 min_in_court_frac=float(video.get("court_min_in_court_frac", 0.5)),
254                 height_pad=float(video.get("court_height_pad", 0.0) or 0.0),
255             )
256         if min_crossings > 0 and ivs:
257             from backend.pipeline.detectors.rally_gate import gate_windows
258
259             gated = [
260                 (g.start, g.end)
261                 for g in gate_windows(
262                     points, ivs, video["fps"], min_crossings=min_crossings
263                 )
264                 if g.kept
265             ]
266             # Gating emptied a non-empty video ? keep the ungated windows.
267             ivs = gated if gated else ivs
268         return ivs
269
270
271         #: One windowed video before scoring: (name, court_type, predicted intervals, GT
intervals).
272         PredictedVideo = Tuple[str, str, List[Interval], List[Interval]]
273
274
275         def predict_all(
276             golden: List[Dict[str, Any]], preset: WindowingPreset, min_crossings: int = 0
277         ) -> List["PredictedVideo"]:
278             """Window every scorable golden video ONCE (the expensive trajectory-parse +
windowing step),
279             returning ``(name, court_type, preds, gts)`` per video. Lets callers re-score the
same
280             predictions under several lenses (e.g. the all-vs-long stratified report) without
re-windowing."""
281             out: List[PredictedVideo] = []
282             for v in golden:
283                 if not v["gts"] or not os.path.exists(v["trajectory"]):
284                     continue
285                 out.append(
286                     (
287                         v["name"],
288                         court_type_for(v),
289                         windows_for(v, preset, min_crossings),
290                         v["gts"],
291                     )
292                 )
293             return out

```

```

294
295
296     def _score_predicted(
297         predicted: List["PredictedVideo"],
298         iou: float,
299         tau_start: Optional[float],
300         tau_end: Optional[float],
301         min_duration: float,
302     ) -> List[Dict[str, Any]]:
303         """Score pre-windowed videos into per-video rows (carrying ``name`` +
304         ``court_type``)."""
305         rows: List[Dict[str, Any]] = []
306         for name, court, preds, gts in predicted:
307             row = score_intervals(preds, gts, iou, tau_start, tau_end, min_duration)
308             row["name"] = name
309             row["court_type"] = court
310             rows.append(row)
311         return rows
312
313     def evaluate(
314         golden: List[Dict[str, Any]],
315         preset: WindowingPreset,
316         iou: float = DEFAULT_IOU,
317         min_crossings: int = 0,
318         tau_start: Optional[float] = None,
319         tau_end: Optional[float] = None,
320         min_duration: float = 0.0,
321     ) -> Dict[str, Any]:
322         """Score every golden video under ``preset`` (+ optional net-crossings gate); return
323         per-video rows + the aggregate. Each row carries both the tIoU set and the R2 tolerance-match
324         block, plus its ``court_type``. ``min_duration`` > 0 applies the long-rally lens (preds + GTs
325         filtered to rallies longer than that many seconds) consistently across both metric paths."""
326         rows = _score_predicted(
327             predict_all(golden, preset, min_crossings),
328             iou,
329             tau_start,
330             tau_end,
331             min_duration,
332         )
333         return {
334             "preset": preset.to_dict(),
335             "iou": iou,
336             "min_crossings": min_crossings,
337             "min_duration": min_duration,
338             "rows": rows,
339             "aggregate": aggregate(rows),
340         }
341
342
343     def compare(
344         golden: List[Dict[str, Any]],
345         base: WindowingPreset,
346         cand: WindowingPreset,
347         iou: float = DEFAULT_IOU,
348         base_min_crossings: int = 0,
349         cand_min_crossings: int = 0,
350         tau_start: Optional[float] = None,
351         tau_end: Optional[float] = None,
352         guard_weight_merge: Optional[float] = None,
353         guard_weight_split: Optional[float] = None,
354         guard_use_rates: bool = True,

```

```

355         min_duration: float = 0.0,
356     ) -> Dict[str, Any]:
357         """Baseline vs candidate (windowing and/or net-crossings gate): per-video paired ?
(+ CI +
358         sign-test) on both tIoU-F1 and the R2 ``tol_f1`` ? the per-tier 'number'. Also
reports ? on the
359         over-segmentation metrics AND the **R1 net over-seg promotion guard**
(``over_seg_guard``):
360         the honest promote/block verdict for cand-over-base, with the strict per-video rule
reported
361         alongside so the refinement is auditable. ``min_duration`` > 0 scores both sides
through the
362         long-rally lens (same filter applied to base and cand ? it's a scoring lens, not a
windowing
363         knob, so it is identical on both sides of the A/B)."""
364         from backend.eval import tolerance_metrics as tm
365
366         b = evaluate(
367             golden, base, iou, base_min_crossings, tau_start, tau_end, min_duration
368         )
369         c = evaluate(
370             golden, cand, iou, cand_min_crossings, tau_start, tau_end, min_duration
371         )
372         by_name_b = {r["name"]: r for r in b["rows"]}
373         paired = [(by_name_b[r["name"]], r) for r in c["rows"] if r["name"] in by_name_b]
374         delta: Dict[str, Any] = {}
375         for k in (
376             "f1",
377             "f1@0.25",
378             "tol_f1",
379             "merge_rate",
380             "split_rate",
381             "segment_count_ratio",
382         ):
383             delta[k] = eval_stats.paired_delta_ci(
384                 [rb[k] for rb, _ in paired], [rc[k] for _, rc in paired]
385             )
386         gkw: Dict[str, Any] = {"use_rates": guard_use_rates}
387         if guard_weight_merge is not None:
388             gkw["weight_merge"] = guard_weight_merge
389         if guard_weight_split is not None:

```

41. assistant (2026-07-03T17:36:20.872Z)

This is critical! Let me look at line 189 again and understand what changed:

```

**OLD behavior in rally_seg_eval.load_golden:**
```python
manifest = json.load(fh) # line 189, originally
```

**NEW behavior:**
```python
manifest = load_resolved_manifest(manifest_path, required_fields=("trajectory",))
```

```

The NEW code loads with `strict=True` (default) and `required\_fields=("trajectory",)`. This means:

- If any video in the manifest is missing a trajectory file ? ManifestPathError ? function raises

Then in line 283:

```

```python
if not v["gts"] or not os.path.exists(v["trajectory"]):
 continue

```



```

804 "exit_code": result.exit_code(),
805 "current": snapshot,
806 "regressions": [
807 f.message() for c in result.comparisons for f in c.regressions
808],
809 },
810 fh,
811 indent=2,
812 sort_keys=True,
813)
814 log.info("report written to %s", md_path)
815
816 print(report)
817 code = result.exit_code()
818 if code == 1:
819 log.error(
820 "NIGHTLY REGRESSION DETECTED (%d) ? see %s",
821 sum(len(c.regressions) for c in result.comparisons),
822 args.report_dir,
823)
824 elif code == 4:
825 log.warning(
826 "baseline STALE ? corpus/config changed; refresh with --update-baseline"
827)
828 else:
829 log.info("nightly regression check PASSED")
830 return code
831
832
833 if __name__ == "__main__":
834 raise SystemExit(main())
835

```

#### 43. assistant (2026-07-03T17:36:23.871Z)

The exception handling at line 761 only catches `RuntimeError`, NOT `ManifestPathError`. Let me check what `ManifestPathError` inherits from:

#### 44. user (2026-07-03T17:36:25.406Z)

```

class ManifestPathError(RuntimeError):
 """Raised when a strict manifest load finds missing required files."""

 def __init__(self, manifest_path: str | os.PathLike[str], issues: Sequence[ManifestIssue]):

```

#### 45. assistant (2026-07-03T17:36:28.261Z)

Great! `ManifestPathError` IS a `RuntimeError`, so it WILL be caught. Now let me look more carefully at the old behavior vs new in all these functions. Let me check if any of these 8 functions had special tolerances before. Let me search for what the old code did in each one:

#### 46. user (2026-07-03T17:36:29.425Z)

```

def run(
 manifest_path: str,
 out_dir: str,
 frame_skip: int = 3,
 iou: float = 0.5,
 fresh: bool = False,
 smallest_first: bool = False,
) -> List[str]:
 with open(manifest_path, encoding="utf-8") as f:
 specs = json.load(f)
 if smallest_first:

```

```

 # Process small videos first for quick feedback (the slow full match lands last);
 # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
 specs = sorted(
 specs,
 key=lambda s: (
 os.path.getsize(s["trajectory"])
 if os.path.isfile(s.get("trajectory", ""))
 else 0
),
)
)

```

#### 47. assistant (2026-07-03T17:36:32.683Z)

Aha! The OLD code in `fusion_golden.run()` did NOT validate that files exist. Let me look at what happens when trajectory doesn't exist:

#### 48. user (2026-07-03T17:36:33.567Z)

```

def run(
 manifest_path: str,
 out_dir: str,
 frame_skip: int = 3,
 iou: float = 0.5,
 fresh: bool = False,
 smallest_first: bool = False,
) -> List[str]:
 with open(manifest_path, encoding="utf-8") as f:
 specs = json.load(f)
 if smallest_first:
 # Process small videos first for quick feedback (the slow full match lands last);
 # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
 specs = sorted(
 specs,
 key=lambda s: (
 os.path.getsize(s["trajectory"])
 if os.path.isfile(s.get("trajectory", ""))
 else 0
),
)
 os.makedirs(out_dir, exist_ok=True)
 # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so keep a tiny
 # rotating runlog of velocity + machine health for the post-mortem. Constructing it must
never
 # break extraction, so guard the build and fall back to no telemetry on any error.
 telem: Optional[RunTelemetry]
 try:
 telem = RunTelemetry(
 os.path.join(out_dir, "run-telemetry.jsonl"), label="fusion_golden"
)
 except Exception: # noqa: BLE001 - telemetry wiring must never break the run it observes
 telem = None
 written: List[str] = []
 for vi, spec in enumerate(specs, 1):
 out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
 # Resumable: skip a video whose JSON already exists (a re-run picks up where a prior
 # run left off instead of re-detecting). --fresh forces a redo.
 if not fresh and os.path.isfile(out):
 print(
 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already extracted "
 f"-> {out} (use --fresh to redo)",
 flush=True,
)
 written.append(out)

```

```

 continue
 print(
 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
 flush=True,
)
 t0 = time.monotonic()
 if telem is not None:
 telem.event("video_start", spec["name"])
 try:
 data = extract_video(spec, frame_skip=frame_skip, iou=iou, telemetry=telem)
 except Exception as e: # noqa: BLE001 - record machine state at death, then re-raise
 if telem is not None:
 telem.event("error", spec["name"], err=str(e))
 raise
 n_pos = sum(c["label"] for c in data["candidates"])
 with open(out, "w", encoding="utf-8") as f:
 json.dump(data, f, indent=2)
 if telem is not None:
 telem.event(
 "video_done",
 spec["name"],
 windows=len(data["candidates"]),
 positives=n_pos,
)
 print(
 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in {time.monotonic() -
t0:.0f}s "
 f"| {len(data['candidates'])} windows ({n_pos} positive) | {len(data['gts'])} golden
rallies "
 f"-> {out}",
 flush=True,
)
 written.append(out)
 return written

def main() -> None:
 ap = argparse.ArgumentParser(
 description="Extract golden-set shuttle+person features (fusion P3 GPU step)."
)
 ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
 ap.add_argument("--out-dir", default="output")
 ap.add_argument("--frame-skip", type=int, default=3)
 ap.add_argument("--iou", type=float, default=0.5)
 ap.add_argument(
 "--fresh", action="store_true", help="re-extract even if a JSON already exists"
)
 ap.add_argument(
 "--smallest-first",
 action="store_true",
 help="process smaller videos first for quick feedback (LOVO is order-independent)",
)
 args = ap.parse_args()
 run(
 args.manifest,
 args.out_dir,
 args.frame_skip,
 args.iou,
 fresh=args.fresh,

```

**49. assistant (2026-07-03T17:36:39.516Z)**

Let me check what extract\_video does with a missing trajectory:

**50. user (2026-07-03T17:36:40.561Z)**

```

def extract_video(
 spec: Dict[str, Any],
 detector: Optional[Any] = None,
 frame_skip: int = 3,
 iou: float = 0.5,
 log_every_sec: float = 15.0,
 telemetry: Optional[Any] = None,
) -> Dict[str, Any]:
 """One golden video -> a feature record (shuttle + person features per candidate window).

 Returns ``{name, fps, frame_width, candidates:[{start,end,shuttle{}},person{},label]},
 gts}``.
 `detector` (anything with ``detections_over_windows``) is injectable for GPU-free tests;
 when None a real `PersonDetector` is built (loads torchvision on first use).
 `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health snapshots
 from the detection progress callback so a hard kill leaves a forensic runlog.
 """
 fps, fw = float(spec["fps"]), float(spec["frame_width"])
 traj = str(spec["trajectory"])
 points = TrackNetRunner.parse_trajectory_csv(traj)
 intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
 gts = load_golden_gts(str(spec["labels"]))
 labels = label_candidates(intervals, gts, iou)
 # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic estimate
 # (parity with the shuttle net-crossing gate) rather than hard-coding an orientation.
 axis, net_px, _span = rally_gate.estimate_play_axis(points)
 video_path = _derive_video_path(spec)

 if detector is None:
 from backend.pipeline.detectors.person_detector import PersonDetector

 detector = PersonDetector({"indexing": {"use_gpu": True}})

 # Single-pass person detection over ALL candidate windows (open the video ONCE, read
 # forward ? no per-window seek). Then compute features per window (pure, no GPU).
 n = len(intervals)
 win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
 print(
 f"[fusion_golden] {spec['name']}: {n} windows ? person-detecting in one pass...",
 flush=True,
)
 t0 = time.monotonic()
 state = {"last": t0}

 def _progress(done: int, total: int) -> None:
 if telemetry is not None:
 telemetry.snapshot(done, total, phase=spec["name"])
 now = time.monotonic()
 if now - state["last"] >= log_every_sec or done >= total:
 el = now - t0
 rate = done / el if el > 0 else 0.0
 eta = (total - done) / rate if rate > 0 else 0.0

```

## 51. assistant (2026-07-03T17:36:45.474Z)

So `extract_video` calls `TrackNetRunner.parse_trajectory_csv(traj)` on line (the exact line not shown, but clearly early). If trajectory file is missing, this will raise an error. So the OLD behavior was:

- Load specs from JSON (all of them)
- Try to extract each one
- If a trajectory is missing, that extraction fails with an error

The NEW behavior is:

- Load specs and VALIDATE that all required files exist upfront
- If any file is missing, raise `ManifestPathError` immediately

This is a behavioral change for `fusion_golden.run()` when called with a partial corpus (some trajectories missing):

- OLD: Would process the ones that exist, fail on the first missing one
- NEW: Would fail immediately on manifest load

Now let me search for any usage patterns in `fusion_golden` that might depend on resumability:

## 52. user (2026-07-03T17:36:46.406Z)

81b0cfc fix(wasb): don't abspath the cache key in manifest\_compatible (#348) (#350)  
2f4ecad docs(runbook): GCS-bucket videos ? highlights on the L4 Cloud Run + committed build config (#346)  
453314f feat(serving): wire /api/process to dispatch DETECT to Cloud Run (M6 ? API), default-OFF (#333)  
df3b225 docs(nightly): one-stop operations manual for the nightly engine-regression (#325)  
a42c132 docs(pkg): track the batteries-included app plan (PACKAGING\_PLAN.md) + index pointers (#309)  
8cdf65c feat(serving): M6 ? Cloud Run GPU dispatch shim + worker image (default-OFF, mocked) (#287)  
55ae408 docs(decoupled): GCP pivot runbook + DO?GCP ADR-010 (deploy/gcp + handoff) (#257)  
dc53db5 feat(detectors): Linux-native WASB runner (M3.5) + worker real-trajectory extraction (#256)  
5c77f48 docs(decoupled): HANDOFF.md ? session checkpoint / resume-here (#255)  
c7df281 perf(wasb): stream video frames to the detector instead of extracting ~46k PNGs (#178)  
e94ef62 docs: shared-data plan + next-session pickup anchor (#175)  
428ee44 address follow-up review on #169: correct factual error in item-3 investigation note (fusion\_compare.py, fusion\_golden.py, ablation.py + tests \*do\* exist per ls/grep on the tree; person features in fusion\_hybrid + detectors/fusion\_features.py; remaining M0 in-container task is wiring into compare). Cleaned duplicate sequencing paragraph. Verified files present. Plan now accurate. M0 item 3 placement remains correct and even stronger.  
918a4ff feat(jobs): worker self-scheduling drain loop honoring WAIT\_AND\_RESUME (EXECUTION\_POLICY P3a) (#149)  
77082f4 feat(eval): wire run-telemetry black box into fusion\_golden extractor (#140)  
2673eee docs: execution-policy framework ? per-job, self-scheduling resilience (design) (#132)  
813d1f8 feat(eval): fusion\_golden --smallest-first (quick-feedback extraction order) (#126)  
351a27e perf(eval): single-pass person detection for fusion\_golden (~10x faster extraction) (#125)  
f4d7008 feat(eval): fusion\_golden progress logging + resumability (observability) (#123)  
dc19bf2 feat(eval): fusion P4 ? audio cue + incremental-lift measurement (default-OFF) (#122)  
39e961b feat(eval): fusion P3 measurement ? golden person-feature extractor + LOVO compare (#121)

## 53. assistant (2026-07-03T17:36:49.290Z)

Let me look at the resume feature in `fusion_golden`:

## 54. user (2026-07-03T17:36:50.382Z)

```
100 now = time.monotonic()
101 if now - state["last"] >= log_every_sec or done >= total:
102 el = now - t0
103 rate = done / el if el > 0 else 0.0
104 eta = (total - done) / rate if rate > 0 else 0.0
105 pct = 100 * done // total if total else 0
106 print(
107 f"[fusion_golden] {spec['name']}: detect frame {done}/{total} ({pct}%)
108 "
109 f"| {el:.0f}s elapsed | ETA {eta:.0f}s",
110 flush=True,
111)
112 state["last"] = now
113
114 per_window = detector.detections_over_windows(
115 video_path, win_frames, frame_skip, progress_cb=_progress
```

```

115)
116
117 candidates: List[Dict[str, Any]] = []
118 for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
119 det = per_window[i]
120 det_fw = det.get("frame_width") or fw
121 det_fh = det.get("frame_height") or 0.0
122 # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
123 # normalisation fusion_features uses (x/width, y/height).
124 if axis == "x":
125 net_norm = net_px / det_fw if det_fw > 0 else 0.5
126 else:
127 net_norm = net_px / det_fh if det_fh > 0 else 0.5
128 sf, ef = win_frames[i]
129 shuttle_in = [p for p in points if sf <= p.frame <= ef]
130 person = ff.compute_person_features(
131 det["frames"],
132 shuttle_in,
133 det_fw,
134 det_fh,
135 court_polygon=None,
136 net=(axis, net_norm),
137)
138 candidates.append(
139 {
140 "start": float(s),
141 "end": float(e),
142 "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
143 "person": {k: float(v) for k, v in person.items()},
144 "label": int(labels[i]),
145 }
146)
147 return {
148 "name": spec["name"],
149 "fps": fps,
150 "frame_width": fw,
151 "candidates": candidates,
152 "gts": [[float(a), float(b)] for a, b in gts],
153 }
154
155
156 def run(
157 manifest_path: str,
158 out_dir: str,
159 frame_skip: int = 3,
160 iou: float = 0.5,
161 fresh: bool = False,
162 smallest_first: bool = False,
163) -> List[str]:
164 specs = load_resolved_manifest(manifest_path)
165 if smallest_first:
166 # Process small videos first for quick feedback (the slow full match lands
last);
167 # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
168 specs = sorted(
169 specs,
170 key=lambda s: (
171 os.path.getsize(s["trajectory"])
172 if os.path.isfile(s.get("trajectory", ""))
173 else 0
174),
175)
176 os.makedirs(out_dir, exist_ok=True)

```

```

177 # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
keep a tiny
178 # rotating runlog of velocity + machine health for the post-mortem. Constructing it
must never
179 # break extraction, so guard the build and fall back to no telemetry on any error.
180 telem: Optional[RunTelemetry]
181 try:
182 telem = RunTelemetry(
183 os.path.join(out_dir, "run_telemetry.jsonl"), label="fusion_golden"
184)
185 except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
observes
186 telem = None
187 written: List[str] = []
188 for vi, spec in enumerate(specs, 1):
189 out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
190 # Resumable: skip a video whose JSON already exists (a re-run picks up where a
prior
191 # run left off instead of re-detecting). --fresh forces a redo.
192 if not fresh and os.path.isfile(out):
193 print(
194 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already extracted
"
195 f"-> {out} (use --fresh to redo)",
196 flush=True,
197)
198 written.append(out)
199 continue
200 print(
201 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
202 flush=True,
203)
204 t0 = time.monotonic()
205 if telem is not None:
206 telem.event("video_start", spec["name"])
207 try:
208 data = extract_video(spec, frame_skip=frame_skip, iou=iou, telemetry=telem)
209 except Exception as e: # noqa: BLE001 - record machine state at death, then re-
raise
210 if telem is not None:
211 telem.event("error", spec["name"], err=str(e))
212 raise
213 n_pos = sum(c["label"] for c in data["candidates"])
214 with open(out, "w", encoding="utf-8") as f:
215 json.dump(data, f, indent=2)
216 if telem is not None:
217 telem.event(
218 "video_done",
219 spec["name"],
220 windows=len(data["candidates"]),
221 positives=n_pos,
222)
223 print(
224 f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
225 f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
226 f"-> {out}",
227 flush=True,
228)
229 written.append(out)
230 return written
231
232
233 def main() -> None:

```

```

234 ap = argparse.ArgumentParser(
235 description="Extract golden-set shuttle+person features (fusion P3 GPU step)."
236)
237 ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
238 ap.add_argument("--out-dir", default="output")
239 ap.add_argument("--frame-skip", type=int, default=3)

```

#### 55. assistant (2026-07-03T17:36:53.044Z)

Now let me check served\_gate\_a.py to see if it has any special resume/tolerance logic:

#### 56. user (2026-07-03T17:36:53.978Z)

```

195
196 def run(
197 manifest_path: str, iou: float = 0.5
198) -> Tuple[List[ServedVideoResult], Dict[str, Any]]:
199 specs = load_resolved_manifest(manifest_path)
200 results = evaluate_manifest(specs, iou=iou)
201 return results, aggregate(results)
202
203
204 def main(argv: Optional[List[str]] = None) -> None:
205 ap = argparse.ArgumentParser(
206 description="Verify served Gate-A on real footage via the production
207 FusionSegmenter path."
208)
209 ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
210 ap.add_argument("--iou", type=float, default=0.5)
211 args = ap.parse_args(argv)
212 results, agg = run(args.manifest, iou=args.iou)
213 print(format_table(results, agg, args.iou))
214
215 if __name__ == "__main__":
216 main()
217

```

#### 57. assistant (2026-07-03T17:36:56.646Z)

Now let me check if any of the removed `import json` lines are critical. Let me focus on served\_gate\_a.py since it removed json:

#### 58. user (2026-07-03T17:36:57.823Z)

```

--- a/backend/eval/served_gate_a.py
+++ b/backend/eval/served_gate_a.py
@@ -23,13 +23,13 @@ Run (owner box, golden trajectories present):
@@ -196,8 +196,7 @@ def format_table(

```

#### 59. assistant (2026-07-03T17:37:00.698Z)

Good, served\_gate\_a.py doesn't use json after the import removal (confirmed earlier). Now let me compile my findings. Let me check one more thing - do any functions use rally\_seg\_eval.load\_golden to see if the strict behavior breaks them:

#### 60. user (2026-07-03T17:37:02.002Z)

```

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/ablation.py:
return experiment.load_golden_videos(
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/audio/e1_probe.py:Reuses: audio.hit_detector,
calibrate_wasb.load_golden_gts/make_predict_fn,
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-

```

```

indexer/backend/eval/audio/e1_probe.py:from backend.eval.calibrate_wasb import BASELINE,
load_golden_gts, make_predict_fn
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/audio/e1_probe.py: gts = load_golden_gts(labels_csv)
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/calibrate_local.py:Reuses: TrackNetRunner windowing, rally_gate,
calibrate_wasb.load_golden_gts,
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/calibrate_local.py:from backend.eval.calibrate_wasb import load_golden_gts
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/calibrate_local.py: gts = load_golden_gts(s["labels"])
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/calibrate_wasb.py:def load_golden_gts(csv_path: str) -> List[Interval]:
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/calibrate_wasb.py: gts = load_golden_gts(labels_csv)
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/distill_local.py:rally_gate, metrics, calibrate_wasb.load_golden_gts,
gemini_refine.merge_overlaps.
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/distill_local.py:from backend.eval.calibrate_wasb import load_golden_gts
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/distill_local.py: gts = load_golden_gts(spec["labels"])
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/eval_partial_labels.py: - calibrate_wasb.make_predict_fn / load_golden_gts
/ BASELINE
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/eval_partial_labels.py:from backend.eval.calibrate_wasb import BASELINE,
load_golden_gts, make_predict_fn
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/eval_partial_labels.py: gts_all = load_golden_gts(labels_csv)
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/eval_partial_labels.py: gts_all = load_golden_gts(labels_csv)
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/experiment.py:def load_golden_videos(
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/export_wasb_segments.py:from backend.eval.calibrate_wasb import BASELINE,
load_golden_gts, make_predict_fn
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/export_wasb_segments.py: gts = load_golden_gts(labels_csv)
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/fusion_audio.py:
return experiment.load_golden_videos(
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/fusion_compare.py: return experiment.load_golden_videos(
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/fusion_golden.py:from backend.eval.calibrate_wasb import load_golden_gts
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/fusion_golden.py: gts = load_golden_gts(str(spec["labels"]))
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/eval/gt_loader.py:-
`calibrate_wasb.load_golden_gts`: DictReader requiring `start_time/end_time`
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/rally_seg_eval.py:def load_golden(
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/rally_seg_eval.py: golden = load_golden(args.manifest,
args.features_dir)
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/rally_seq_proto.py:from backend.eval.calibrate_wasb import load_golden_gts
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/rally_seq_proto.py: gts = load_golden_gts(s["labels"])
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/served_gate_a.py:from backend.eval.calibrate_wasb import load_golden_gts
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/served_gate_a.py: gts = load_golden_gts(labels)
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/backend/eval/serve_contrast.py:from backend.eval.calibrate_wasb import load_golden_gts
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-

```

```

indexer/backend/eval/serve_contrast.py: gts = load_golden_gts(str(spec["labels"]))
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/archives/quality-
iterations/2026-06-18-lovo-resweep-n13/lovo_resweep.py:from backend.eval.rally_seg_eval import
load_golden, windows_for # noqa: E402
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/archives/quality-
iterations/2026-06-18-lovo-resweep-n13/lovo_resweep.py: golden = load_golden(MANIFEST,
FEATURES)
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/scratch/ab_n6_lovo.py:from
backend.eval.calibrate_wasb import load_golden_gts
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/scratch/ab_n6_lovo.py:
gts = load_golden_gts(s["labels"])
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/scratch/eval_multitrack_gx128_golden.py:def load_golden(path):
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/scratch/eval_multitrack_gx128_golden.py: gts = load_golden(GOLDEN)
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/scratch/measure_gate_a.py:from backend.eval.calibrate_wasb import load_golden_gts
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/scratch/measure_gate_a.py:
gts = load_golden_gts(s["labels"])
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/scratch/measure_served_gate_a_15clip.py:from backend.eval.calibrate_wasb import
load_golden_gts
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/scratch/measure_served_gate_a_15clip.py: gts = load_golden_gts(s["labels"])
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/scratch/r1_evidence.py:
golden = rse.load_golden(MANIFEST, OUT)
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/scratch/r5_evidence.py:
golden = rse.load_golden(MANIFEST, OUT)
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/scratch/served_gateA_sweep.py:from backend.eval.calibrate_wasb import load_golden_gts
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/scratch/served_gateA_sweep.py: gts = load_golden_gts(s["labels"])
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/scratch/served_gateA_verify.py:from backend.eval.calibrate_wasb import load_golden_gts
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/scratch/served_gateA_verify.py: gts = load_golden_gts(spec["labels"])
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/scripts/nightly_regression.py: golden = rally_seg_eval.load_golden(manifest,
features_dir)

```

## 61. assistant (2026-07-03T17:37:06.829Z)

Good, `rally\_seg\_eval.load\_golden` is called in scripts/nightly\_regression.py and in docs/archives/... Now let me wrap up my analysis. Let me create a comprehensive list of the issues I've found:

## 62. user (2026-07-03T17:37:13.756Z)

### REGRESSION CANDIDATES SUMMARY:

- test\_served\_gate\_a\_regression.py: \_golden\_specs() behavior change
  - OLD: Filtered specs to keep only those with both trajectory AND labels files present
  - NEW: Strict load ALL specs, raise if ANY file missing, catch exception
  - RISK: A box with partial corpus (missing some trajectories/labels) that previously ran >=6 specs will now skip the whole test
- rally\_seg\_eval.load\_golden(): strict manifest loading
  - OLD: json.load(fh) - accepts any specs, loads features selectively
  - NEW: load\_resolved\_manifest(..., required\_fields=("trajectory",)) - raises if ANY trajectory missing
  - IMPACT: scripts/nightly\_regression.py calls this; if corpus is incomplete, will get ManifestPathError (caught as RuntimeError)
- fusion\_golden.run(): strict manifest loading with resume capability
  - OLD: json.load(f) - loads all specs, processes available ones

- NEW: load\_resolved\_manifest(manifest\_path) - raises if any trajectory/labels missing
  - RISK: --fresh resume flow breaks if some files missing (OLD: would skip the missing ones, NEW: fails at load)
- Multiple test files (test\_fusion\_compare.py, test\_fusion\_golden\_telemetry.py):
    - NOW CREATE missing trajectory/labels files to satisfy strict loader
    - LOSS: Tests no longer validate that the functions can TOLERATE missing files gracefully
    - Changed assertion target: used to test partial-corpus behavior, now forces complete corpus
  - calibrate\_local.py, distill\_local.py, rally\_seq\_proto.py, serve\_contrast.py, served\_gate\_a.py, gen0/harness.py:
    - All changed from json.load() to load\_resolved\_manifest()
    - All will now fail upfront if required files missing (OLD: would fail lazily during processing)
  - JSON import removals:
    - Removed from: calibrate\_local.py, distill\_local.py, rally\_seq\_proto.py, served\_gate\_a.py
    - fusion\_golden.py, rally\_seq\_eval.py still import json (not removed, OK)
    - gen0/harness.py still uses json.load(floor\_path) and json.dump, import was added back implicitly? CHECK

### 63. assistant (2026-07-03T17:37:15.675Z)

Let me verify the gen0/harness.py import issue:

### 64. user (2026-07-03T17:37:16.454Z)

```

1 """Gen-0 TAL harness (M0.4): ONE command -> train -> honest LOVO eval -> ship-gate ->
artifact.
2
3 python -m training.gen0.harness --manifest output/human_lovo_manifest.json \
4 --floor eval_baselines/heuristic_lovo_n6.json --out artifacts/rally_tal_gen0
--version 0.1.0
5
6 Productionizes the rally_seq prototype per ADR-008:
7 - TRAINS on the corpus manifest (per-video trajectory + golden labels), evaluates with
8 honest leave-one-video-out (split by match, threshold tuned on train only), then fits
9 the FINAL model on all videos.
10 - Emits a versioned ARTIFACT BUNDLE ? the only thing serving may ever consume:
11 <out>/<version>/weights.json (gitignored; classifier + frozen normalization)
12 <out>/<version>/manifest.json (the reviewable contract: feature-schema version,
13 calibration, training lineage + GT hashes,
14 eval results, attestations, gate verdict)
15 - Runs the SHIP-GATE as pre-flight: floor comparison + a PAIRED per-video sign test
16 (at n=6 a mean gap alone is noise-ambiguous), plus the hard attestation blocks
17 (WASB C3 unresolved => commercial ship refused; explicit F1@tIoU target still
18 undefined => no self-declared victory). The product side owns final admission
19 (backend/eval/regression.py); this harness never grades itself into serving.
20
21 Training may import backend.* (one-way wall); backend must never import training.*.
22 """
23
24 from __future__ import annotations
25
26 import argparse
27 import hashlib
28 import json
29 import os
30 import re
31 import subprocess
32 import time
33 from math import comb
34 from typing import Dict, List, Optional
35
36 import numpy as np

```

```

37
38 from backend.eval.classifier import LogisticRegression
39 from backend.eval.golden_manifest import load_resolved_manifest
40 from backend.eval.gt_loader import load_gt_intervals
41 from backend.eval.rally_seq_proto import (
42 _seg_f1,
43 labels_for,
44)
45 from backend.eval.rally_seq_proto import (
46 run as lovo_run,
47)
48 from backend.eval.regression import gt_hash
49 from backend.features.rally_seq import (
50 FEAT_NAMES,

```

## 65. assistant (2026-07-03T17:37:30.480Z)

Good! Line 28 shows `import json` IS present in gen0/harness.py, so no import issue there. Now let me finalize my analysis and produce the JSON output:

Based on my thorough analysis of the PR diff, here are the removed-behavior regressions I found:

```

```json
[
  {
    "file": "tests/test_served_gate_a_regression.py",
    "line": 69,
    "summary": "_golden_specs() changed from per-spec filtering to strict all-or-nothing loading. OLD code kept subset of specs whose files exist and ran if >=6 survived; NEW code strict-loads ALL specs, raises ManifestPathError on ANY missing file, catches exception and returns [].",
    "failure_scenario": "A CI box or partial-corpus environment with 10+ golden videos but 3-4 missing trajectory/labels files: OLD code would filter and run >=6 tests, NEW code skips entire suite when load fails. Coverage regression for partial-corpus validation."
  },
  {
    "file": "backend/eval/rally_seq_eval.py",
    "line": 189,
    "summary": "load_golden() now calls load_resolved_manifest(..., required_fields=(\"trajectory\"),) with strict=True, raising ManifestPathError if ANY trajectory is missing. OLD code json.load(fh) accepted any specs and loaded features selectively per-video.",
    "failure_scenario": "nightly_regression.py calls load_golden() at line 631; if corpus has 1+ missing trajectories (e.g., a stale manifest entry), OLD behavior would skip that video, NEW behavior raises ManifestPathError (caught as RuntimeError at line 761) and returns exit 2. A partial corpus that previously passed nightly check will now fail the entire run."
  },
  {
    "file": "backend/eval/fusion_golden.py",
    "line": 164,
    "summary": "run() replaced json.load(f) with load_resolved_manifest(manifest_path) (strict, default=True). OLD code loaded all specs and processed available ones; resume-capability (--fresh flag, line 192) allowed partial extraction when some files missing. NEW code fails upfront if ANY trajectory/labels file missing.",
    "failure_scenario": "GPU extraction run with --fresh resume on a corpus where 1-2 new videos lack trajectory CSVs: OLD code would skip them and resume on the rest, NEW code raises ManifestPathError at manifest load, aborting the entire resumable batch."
  },
  {
    "file": "tests/test_fusion_compare.py",
    "line": 203,
    "summary": "test_fusion_golden_run_is_resumable() NOW CREATES trajectory/labels files (a.csv, b.csv, a.rallies.csv, b.rallies.csv) that didn't exist before. Test no longer validates the actual resume behavior on a partial corpus; it forces a complete manifest.",
    "failure_scenario": "Test assertion changed: OLD test exercised fusion_golden.run()

```

tolerating missing files per-spec; NEW test fabricates complete file set, so the missing-file tolerance code path (if any) is never exercised. Loss of regression coverage for partial-corpus resume flows."

```
    },
    {
        "file": "tests/test_fusion_golden_telemetry.py",
        "line": 48,
        "summary": "_write_manifest() NOW CREATES trajectory + labels files (n.csv, n.rallies.csv) before writing the manifest JSON. OLD behavior did not create these files, so tests ran on a partial-file corpus.",
        "failure_scenario": "Tests no longer validate fusion_golden's tolerance of missing-file scenarios. If a future change makes the code rely on graceful degradation for missing trajectories, this test suite will not catch the regression."
    },
    {
        "file": "backend/eval/calibrate_local.py",
        "line": 167,
        "summary": "main() replaced json.load(f) with load_resolved_manifest(args.manifest). No special tolerances removed, but now fails upfront on any missing trajectory/labels file instead of failing lazily during calibration_eval.run() processing.",
        "failure_scenario": "A development box running calibrate_local with a stale/incomplete manifest: OLD code would begin processing and fail on first missing file inside run(), NEW code fails at manifest load with ManifestPathError. Error surface changes from 'file not found in run()' to 'manifest path check failed'."
    }
]
...
```